MotionControl_Simu

Near-wall motion control simulation of a magnetically driven scanning microprobe in aqueous solutions. Implements a 23-state Extended Kalman Filter (EKF) for real-time estimation of the position-dependent drag coefficient (motion gain) and wall orientation, enabling ultra-precise trajectory tracking despite the strongly nonlinear wall effect on Stokes drag.

Based on: Meng, Long & Menq, "Near-Wall Ultra-precise Motion Control of a Magnetically Driven Scanning Microprobe in Aqueous Solutions" (IEEE T-ASE). Stack: MATLAB R2025b + Simulink.

Quick Start

Requirements: MATLAB R2025b with Simulink.

Run:

1. Open `test_script/run_simulation.m` in MATLAB.
2. Adjust parameters in SECTION 1 (see table below).
3. Run the script. A tabbed figure opens with results; PNGs and `.mat` data are saved to `test_results/simulation/sim_<timestamp>/`.

Configurable Parameters

Category	Parameter	Default	Description
Wall Geometry	`theta`	0 deg	Wall azimuth angle
Wall Geometry	`phi`	0 deg	Wall elevation angle
Wall Geometry	`pz`	0 um	Wall displacement along normal
Wall Geometry	`h_min`	1.1*R um	Minimum safe wall distance
Trajectory	`h_init`	5 um	Initial distance from wall
Trajectory	`amplitude`	2.5 um	Oscillation amplitude along wall normal
Trajectory	`frequency`	1 Hz	Oscillation frequency
Trajectory	`n_cycles`	3	Number of oscillation cycles
Controller	`ctrl_enable`	true	Enable closed-loop control (false = open-loop)
Controller	`lambda_c`	0.7	Closed-loop pole (0 < lambda_c < 1)
EKF	`a_pd`	0.1	IIR coefficient: deterministic component
EKF	`a_prd`	0.1	IIR coefficient: random-deterministic component
EKF	`a_cov`	0.1	IIR coefficient: covariance estimation
EKF	`epsilon`	0.01	Anisotropy threshold for wall angle measurement
Noise	`meas_noise_enable`	true	Inject measurement noise
Noise	`meas_noise_std`	[0.00062; 0.000057; 0.00331] um	Per-axis measurement noise std
Noise	`thermal_enable`	true	Enable Brownian thermal force

Output Tabs

Closed-loop (8 tabs): 3D Trajectory, X/Y/Z-Axis tracking, Control Force, Lambda Estimation, Theta Estimation, Error Statistics.

Open-loop (7 tabs): 3D Trajectory, X/Y/Z-Axis tracking, Control Force, OL Time Response, OL FFT Spectrum.

Project Structure

```
MotionControl_Simu/
├── model/
│   ├── calc_simulation_params.m        # Master parameter calculator + Bus Object definitions
│   ├── system_model.slx                # Simulink main model
│   ├── config/
│   │   ├── user_config.m               # User-adjustable parameter defaults
│   │   └── physical_constants.m        # Physical constants (R, gamma_N, k_B, T, Ts)
```

```
│   ├── controller/
│   │   ├── motion_control_law.m        # EKF estimator + control law (main logic)
│   │   └── calc_ctrl_params.m          # Derived controller parameters
│   ├── wall_effect/
│   │   ├── calc_wall_params.m          # Wall geometry from (theta, phi)
│   │   ├── calc_correction_functions.m # Drag correction c_para(h_bar), c_perp(h_bar)
│   │   └── calc_gamma_inv.m            # Mobility matrix Gamma_inv(p)
│   ├── trajectory/
│   │   ├── trajectory_generator.m      # Sinusoidal trajectory along wall normal
│   │   ├── calc_traj_params.m          # Trajectory parameter packaging
│   │   └── calc_initial_position.m     # Initial position p0 computation
│   │   └── check_trajectory_safety.m   # Pre-flight safety check
│   └── thermal_force/
│       ├── calc_thermal_force.m        # Brownian force generator
│       └── calc_thermal_params.m       # Thermal parameter packaging
├── test_script/
│   └── run_simulation.m                # Main simulation entry point
├── agent_docs/                         # Detailed technical documentation
│   ├── simulink-architecture.md        # Simulink block diagram and solver settings
│   ├── math-model.md                   # Coordinate system, units, system equations
│   ├── ekf-matrix-guide.md             # EKF matrix dimensions and block structure
│   ├── ekf-qr-analysis.md             # EKF Q/R tuning analysis
│   └── analysis-guide.md              # GUI tabs and recommended test parameters
├── reference/
│   ├── controller/
│   │   └── Estimator_and_Control__.pdf # EKF + control law derivation (primary reference)
│   └── thesis/                         # Published thesis PDFs
└── test_results/                       # Simulation output (git-ignored)
```

Controller Development Guide

File Map

File	Role
`model/controller/motion_control_law.m`	EKF estimator + control law – all per-step logic
`model/controller/calc_ctrl_params.m`	Derived parameters (sigma2_deltaXT, g_cov)
`model/config/user_config.m`	User-adjustable parameter defaults
`model/config/physical_constants.m`	Physical constants (R=2.25 um, gamma_N=0.0425 pN*sec/um, Ts=1/1600 sec)

Execution Order per Step [k]

The control law executes **before** the EKF update (standard output-then-update pattern, matching the PDF).

Step	Operation
[0]	Initialize persistent states (first call only)
[1]	Transform desired increment to wall frame: `V_del_pd = V_T * del_pd`
[2]	Compute control force using `lamda_hat`, `V_del_p3_hat`, `V_d_hat`
[3]	Measurement processing: 2-step delay, IIR decomposition, lambda_m, theta_m
[4]	Compute error signals: `err = [err_p; err_lamda; err_theta]` (7x1)
[5]	Kalman gain: `L = Pf * H' * inv(H*Pf*H' + R)` (23x7)
[6]	State update: 9 groups in wall frame, then convert to world frame
[7]	Posterior covariance: `P = (I - L*H) * Pf`
[8]	Build F[k] using G_lambda, G_theta from current step's del_u/v/w
[9]	Build Q[k] using updated lambda_hat
[10]	Forecast covariance: `Pf = F*P*F' + Q`
[11]	Update rotation V[k] from theta_hat; re-project states to new wall frame

Function Interface

```
[f_d, ekf_out] = motion_control_law(del_pd, pd, p_m, params)
```

Inputs:

Argument	Size	Description
`del_pd`	3x1	Trajectory increment `p_d[k+1] - p_d[k]` [um]
`pd`	3x1	Current desired position `p_d[k]` [um]
`p_m`	3x1	Measured position `p_m[k]` [um]
`params`	struct	Parameter structure from `calc_simulation_params`

Outputs:

Argument	Size	Description
`f_d`	3x1	Control force [pN] (world frame)
`ekf_out`	4x1	Diagnostic: `[lamda_hat(2); theta_hat(2)]`

Variable Naming Conventions

Naming Rules

Prefix / Suffix	Meaning	Example
`V_`	Wall-aligned frame (u, v, w)	`V_del_p3_hat`
(none)	World frame (x, y, z)	`del_p3_hat`
`_hat`	Estimated value	`lamda_hat`
`del_`	Increment or difference	`del_pd`
`_m`	Measured value	`lamda_m`
`_kA1`	Value at step [k+1] (during update)	`lamda_hat_kA1`
`_k1`, `_k2`	Delay buffer at [k-1], [k-2]	`pd_k1`, `pd_k2`
`_d`	Desired / reference value	`p_d`, `del_pd`

Key Variable Reference

EKF State Vector (9 groups, 23 dimensions)

Group	Thesis Symbol	MATLAB Name	Size	Description
1	delta_x_hat_1[k]	`V_del_p1_hat`	3x1	Position error delay-1 (wall frame)
2	delta_x_hat_2[k]	`V_del_p2_hat`	3x1	Position error delay-2 (wall frame)
3	delta_x[k]	`V_del_p3_hat`	3x1	Motion error – current tracking error (wall frame)
4	x_D[k]	`V_d_hat`	3x1	Disturbed motion estimate (wall frame)
5	delta_x_D[k]	`V_del_d_hat`	3x1	Disturbed motion rate (wall frame)
6	a_x[k] = 1/c	`lamda_hat`	2x1	Motion gain [tangential; normal]
7	delta_a_x[k]	`del_lamda_hat`	2x1	Motion gain rate of change
8	theta[k]	`theta_hat`	2x1	Wall orientation [theta_x; theta_y] (rad)
9	delta_theta[k]	`del_theta_hat`	2x1	Wall orientation rate of change

Measurement and Error

Thesis Symbol	MATLAB Name	Size	Description
delta_p_m[k]	`del_pm`	3x1	Position error measurement (world), uses 2-step delay

Thesis Symbol	MATLAB Name	Size	Description
–	`del_pmd`	3x1	Deterministic component (IIR filtered, coeff a_pd)
–	`del_pmr`	3x1	Random component
–	`del_pmrd`	3x1	Random-deterministic component (IIR filtered, coeff a_prd)
–	`del_pmrr`	3x1	Residual (used for covariance estimation)
–	`del_pnr`	3x1	Normalized residual (divided by g_cov)
a_m[k]	`lamda_m`	2x1	Motion gain measurement
theta_m[k]	`theta_m`	2x1	Wall angle measurement
e_p[k]	`err_p`	3x1	Position error signal
e_a[k]	`err_lamda`	2x1	Motion gain error signal
e_theta[k]	`err_theta`	2x1	Wall angle error signal

Control Law

Thesis Symbol	MATLAB Name	Size	Description
Delta_u/v/w[k]	`del_u`, `del_v`, `del_w`	scalar	Position increments in wall frame
f_u/v/w[k]	`fu`, `fv`, `fw`	scalar	Control force components (wall frame)
f_d[k]	`f_d`	3x1	Control force output (world frame) [pN]
f_T[k]	`f_th`	3x1	Thermal (Brownian) force [pN]
lambda_c	`lambda_c`	scalar	Closed-loop pole (design parameter)

EKF Matrices

Symbol	MATLAB Name	Size	Description
F[k]	`F`	23x23	State transition matrix (updated per step)
H	`H`	7x23	Observation matrix (fixed)
P	`P`	23x23	Posterior error covariance
P^f	`Pf`	23x23	Forecast (prior) error covariance
Q[k]	`Q`	23x23	Process noise covariance (depends on lamda_hat)
R	`R`	7x7	Measurement noise covariance
L[k]	`L`	23x7	Kalman gain
V[k]	`V`	3x3	Wall rotation matrix (from theta_hat)
G_lambda[k]	`G_lamda`	3x2	Drag coefficient coupling in F
G_theta[k]	`G_theta`	3x2	Wall angle coupling in F

Physical Constants

Symbol	MATLAB Name	Value	Unit
R	`R`	2.25	um
gamma_N	`gamma_N`	0.0425	pN*sec/um
dt	`Ts`	1/1600	sec
k_B	`k_B`	1.3806503e-5	pN*um/K
T	`T`	310.15	K (37 C)

Terminology Alignment (Thesis to Code)

Thesis Term	Code Variable	Notes
motion gain a_x[k]	`lamda_hat`	a_x = (dt / gamma_N) * lambda; lambda = 1/c

Thesis Term	Code Variable	Notes
drag coefficient c(z)	`c_para`, `c_perp`	From `calc_correction_functions.m`
motion error delta_x[k]	`V_del_p3_hat` (Group 3)	3D position tracking error in wall frame
disturbed motion x_D[k]	`V_d_hat` (Group 4)	Disturbance estimate in wall frame
control objective	`lambda_c`	delta_x[k+1] = lambda_c * delta_x[k]
thermal force f_T	`f_th`	Brownian motion force
sampling interval dt	`Ts`	1/1600 sec
measurement delay	`pd_k1`, `pd_k2`	Two-step delay buffer for p_d
IIR filter (a_pd, a_prd)	EMA coefficients `a_pd`, `a_prd`	Deterministic / random separation
scanning microprobe	particle / bead	R = 2.25 um

References

- Meng, Long & Menq, "Near-Wall Ultra-precise Motion Control of a Magnetically Driven Scanning Microprobe in Aqueous Solutions" (IEEE Transactions on Automation Science and Engineering)
- `reference/controller/Estimator_and_Control__.pdf` – EKF and control law derivation
- `agent_docs/ekf-matrix-guide.md` – Full EKF matrix dimensions and block structure reference
- `agent_docs/simulink-architecture.md` – Simulink block diagram, solver settings, data logging
- `agent_docs/math-model.md` – Coordinate system, units, and system equations